



Estácio

Trabalho Prático

Desenvolvimento Back-End Corporativo com Java E Cloud

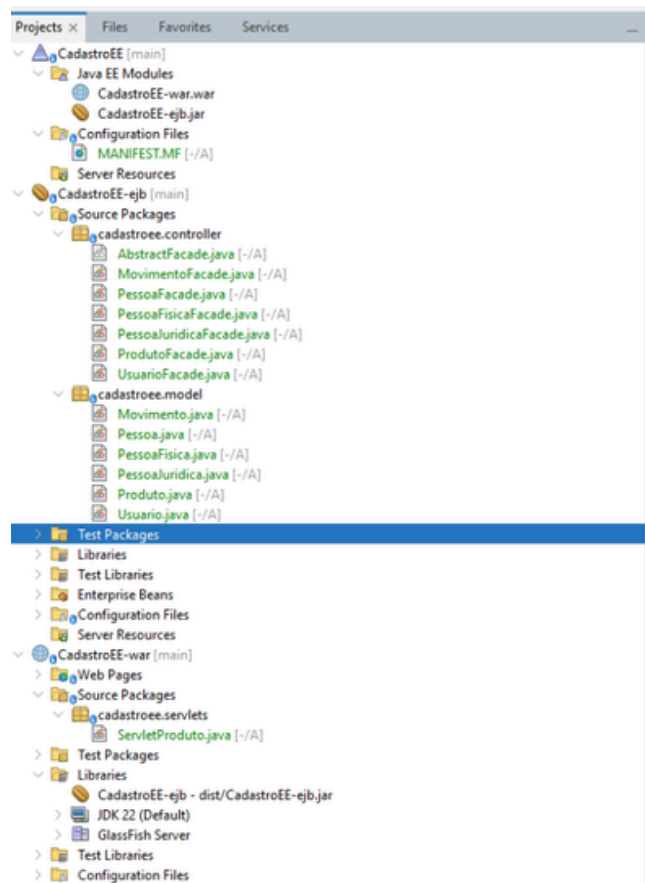
Aluno: Adrian Guilherme Grangeiro Cavalcante 202402131931

Objetivo do Trabalho Prático

O presente trabalho tem como finalidade desenvolver e configurar uma aplicação corporativa baseada na plataforma Java EE utilizando o ambiente NetBeans. A proposta consiste em integrar a camada de persistência de dados, implementada por meio da JPA (Java Persistence API), com a camada de regras de negócio construída utilizando EJBs (Enterprise Java Beans), permitindo a comunicação eficiente com um banco de dados SQL Server.

Além disso, o projeto contempla a criação da camada Web através de Servlets, responsáveis por receber e processar as solicitações dos usuários, realizando a interação com os dados armazenados no banco. Durante a execução da prática, são abordados conceitos relacionados ao ciclo de vida de aplicações corporativas executadas em servidores de aplicação, como o GlassFish, incluindo a configuração de conexões JDBC, gerenciamento de dependências, implantação da aplicação e exibição dinâmica das informações por meio do navegador.

Desenvolvimento



```
Movimento.java [-/M] x
Source History
22  /**
23   *
24   * @author erica
25   */
26  @Entity
27  @Table(name = "Movimento")
28  @XmlRootElement
29  @NamedQueries({
30      @NamedQuery(name = "Movimento.findAll", query = "SELECT m FROM Movimento m"),
31      @NamedQuery(name = "Movimento.findByIdMovimento", query = "SELECT m FROM Movimento m WHERE m.idMovimento = :idMovimento"),
32      @NamedQuery(name = "Movimento.findByQuantidade", query = "SELECT m FROM Movimento m WHERE m.quantidade = :quantidade"),
33      @NamedQuery(name = "Movimento.findByTipo", query = "SELECT m FROM Movimento m WHERE m.tipo = :tipo"),
34      @NamedQuery(name = "Movimento.findByValorUnitario", query = "SELECT m FROM Movimento m WHERE m.valorUnitario = :valorUnitario"))
35  public class Movimento implements Serializable {
36
37      private static final long serialVersionUID = 1L;
38      @Id
39      @GeneratedValue(strategy = GenerationType.IDENTITY)
40      @Basic(optional = false)
41      @Column(name = "idMovimento")
42      private Integer idMovimento;
43      @Column(name = "quantidade")
44      private Integer quantidade;
45      @Column(name = "tipo")
46      private Character tipo;
47
48      @Column(name = "valorUnitario")
49      private BigDecimal valorUnitario;
50      @JoinColumn(name = "idPessoa", referencedColumnName = "idPessoa")
51      @ManyToOne
52      private Pessoa idPessoa;
53      @JoinColumn(name = "idProduto", referencedColumnName = "id")
54      @ManyToOne
55      private Produto idProduto;
56      @JoinColumn(name = "idUsuario", referencedColumnName = "login")
57      @ManyToOne
58      private Usuario idUsuario;
59
60      public Movimento() {
61      }
62
63      public Movimento(Integer idMovimento) {
64          this.idMovimento = idMovimento;
65      }
```

```
65      }
66
67      public Integer getIdMovimento() {
68          return idMovimento;
69      }
70
71      public void setIdMovimento(Integer idMovimento) {
72          this.idMovimento = idMovimento;
73      }
74
75      public Integer getQuantidade() {
76          return quantidade;
77      }
78
79      public void setQuantidade(Integer quantidade) {
80          this.quantidade = quantidade;
81      }
82
83      public Character getTipo() {
84          return tipo;
85      }
86
87      public void setTipo(Character tipo) {
88          this.tipo = tipo;
89      }
90
91      public BigDecimal getValorUnitario() {
92          return valorUnitario;
93      }
94
95      public void setValorUnitario(BigDecimal valorUnitario) {
96          this.valorUnitario = valorUnitario;
97      }
98
99      public Pessoa getIdPessoa() {
100          return idPessoa;
101      }
102
103      public void setIdPessoa(Pessoa idPessoa) {
104          this.idPessoa = idPessoa;
105      }
106
```

```

106
107 public Produto getIdProduto() {
108     return idProduto;
109 }
110
111 public void setIdProduto(Produto idProduto) {
112     this.idProduto = idProduto;
113 }
114
115 public Usuario getIdUsuario() {
116     return idUsuario;
117 }
118
119 public void setIdUsuario(Usuario idUsuario) {
120     this.idUsuario = idUsuario;
121 }
122
123 @Override
124 public int hashCode() {
125     int hash = 0;
126     hash += (idMovimento != null ? idMovimento.hashCode() : 0);
127     return hash;
128 }
129
130 @Override
131 public boolean equals(Object object) {
132     if (!(object instanceof Movimento)) {
133         return false;
134     }
135     Movimento other = (Movimento) object;
136     if ((this.idMovimento == null && other.idMovimento != null) || (this.idMovimento != null && !this.idMovimento.equals(other.idMovimento))) {
137         return false;
138     }
139     return true;
140 }
141
142 @Override
143 public String toString() {
144     return "cadastroee.model.Movimento[ idMovimento=" + idMovimento + " ]";
145 }
146
147 }
148

```

Pessoa.java [-/A] x

Source History

```

1  /*
2   * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
3   * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4   */
5  package cadastroee.model;
6
7  import jakarta.persistence.Basic;
8  import jakarta.persistence.Column;
9  import jakarta.persistence.Entity;
10 import jakarta.persistence.GeneratedValue;
11 import jakarta.persistence.GenerationType;
12 import jakarta.persistence.Id;
13 import jakarta.persistence.NamedQueries;
14 import jakarta.persistence.NamedQuery;
15 import jakarta.persistence.OneToMany;
16 import jakarta.persistence.Table;
17 import jakarta.validation.constraints.Size;
18 import jakarta.xml.bind.annotation.XmlRootElement;
19 import jakarta.xml.bind.annotation.XmlTransient;
20 import java.io.Serializable;
21 import java.util.Collection;
22
23 /**
24  *
25  * @author erica
26  */
27 @Entity
28 @Table(name = "Pessoa")
29 @XmlRootElement
30 @NamedQueries({
31     @NamedQuery(name = "Pessoa.findAll", query = "SELECT p FROM Pessoa p"),
32     @NamedQuery(name = "Pessoa.findByIdPessoa", query = "SELECT p FROM Pessoa p WHERE p.idPessoa = :idPessoa"),
33     @NamedQuery(name = "Pessoa.findByName", query = "SELECT p FROM Pessoa p WHERE p.nome = :nome"),
34     @NamedQuery(name = "Pessoa.findByLogradouro", query = "SELECT p FROM Pessoa p WHERE p.logradouro = :logradouro"),
35     @NamedQuery(name = "Pessoa.findByCidade", query = "SELECT p FROM Pessoa p WHERE p.cidade = :cidade"),
36     @NamedQuery(name = "Pessoa.findByEstado", query = "SELECT p FROM Pessoa p WHERE p.estado = :estado"),
37     @NamedQuery(name = "Pessoa.findByTelefone", query = "SELECT p FROM Pessoa p WHERE p.telefone = :telefone"),
38     @NamedQuery(name = "Pessoa.findByEmail", query = "SELECT p FROM Pessoa p WHERE p.email = :email"))
39 public class Pessoa implements Serializable {
40

```

```

39 public class Pessoa implements Serializable {
40
41     private static final long serialVersionUID = 1L;
42     @Id
43     @GeneratedValue(strategy = GenerationType.IDENTITY)
44     @Basic(optional = false)
45     @Column(name = "idPessoa")
46     private Integer idPessoa;
47     @Size(max = 255)
48     @Column(name = "nome")
49     private String nome;
50     @Size(max = 255)
51     @Column(name = "logradouro")
52     private String logradouro;
53     @Size(max = 255)
54     @Column(name = "cidade")
55     private String cidade;
56     @Size(max = 2)
57     @Column(name = "estado")
58     private String estado;
59     @Size(max = 20)
60     @Column(name = "telefone")
61     private String telefone;
62     @Size(max = 255)
63     @Column(name = "email")
64     private String email;
65     @OneToMany(mappedBy = "idPessoa")
66     private Collection<PessoaJuridica> pessoaJuridicaCollection;
67     @OneToMany(mappedBy = "idPessoa")
68     private Collection<PessoaFisica> pessoaFisicaCollection;
69     @OneToMany(mappedBy = "idPessoa")
70     private Collection<Movimento> movimentoCollection;
71
72     public Pessoa() {
73     }
74
75     public Pessoa(Integer idPessoa) {
76         this.idPessoa = idPessoa;
77     }
78

```

```
78
79 public Integer getIdPessoa() {
80     return idPessoa;
81 }
82
83 public void setIdPessoa(Integer idPessoa) {
84     this.idPessoa = idPessoa;
85 }
86
87 public String getNome() {
88     return nome;
89 }
90
91 public void setNome(String nome) {
92     this.nome = nome;
93 }
94
95 public String getLogradouro() {
96     return logradouro;
97 }
98
99 public void setLogradouro(String logradouro) {
100     this.logradouro = logradouro;
101 }
102
103 public String getCidade() {
104     return cidade;
105 }
106
107 public void setCidade(String cidade) {
108     this.cidade = cidade;
109 }
110
111 public String getEstado() {
112     return estado;
113 }
114
115 public void setEstado(String estado) {
116     this.estado = estado;
117 }
```

```

117     }
118
119     public String getTelefone() {
120         return telefone;
121     }
122
123     public void setTelefone(String telefone) {
124         this.telefone = telefone;
125     }
126
127     public String getEmail() {
128         return email;
129     }
130
131     public void setEmail(String email) {
132         this.email = email;
133     }
134
135     @XmlTransient
136     public Collection<PessoaJuridica> getPessoaJuridicaCollection() {
137         return pessoaJuridicaCollection;
138     }
139
140     public void setPessoaJuridicaCollection(Collection<PessoaJuridica> pessoaJuridicaCollection) {
141         this.pessoaJuridicaCollection = pessoaJuridicaCollection;
142     }
143
144     @XmlTransient
145     public Collection<PessoaFisica> getPessoaFisicaCollection() {
146         return pessoaFisicaCollection;
147     }
148
149     public void setPessoaFisicaCollection(Collection<PessoaFisica> pessoaFisicaCollection) {
150         this.pessoaFisicaCollection = pessoaFisicaCollection;
151     }
152
153     @XmlTransient
154     public Collection<Movimento> getMovimentoCollection() {
155         return movimentoCollection;
156     }
157

```

```

157     public void setMovimentoCollection(Collection<Movimento> movimentoCollection) {
158         this.movimentoCollection = movimentoCollection;
159     }
160
161     @Override
162     public int hashCode() {
163         int hash = 0;
164         hash += (idPessoa != null ? idPessoa.hashCode() : 0);
165         return hash;
166     }
167
168     @Override
169     public boolean equals(Object object) {
170         if (!(object instanceof Pessoa)) {
171             return false;
172         }
173         Pessoa other = (Pessoa) object;
174         if ((this.idPessoa == null && other.idPessoa != null) || (this.idPessoa != null && !this.idPessoa.equals(other.idPessoa))) {
175             return false;
176         }
177         return true;
178     }
179
180     @Override
181     public String toString() {
182         return "cadastroee.model.Pessoa[ idPessoa=" + idPessoa + " ]";
183     }
184
185 }
186
187

```

```
PessoaFisica.java [~/A] x
Source History

1  /*
2   * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
3   * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4   */
5   package cadastroee.model;
6
7   import jakarta.persistence.Basic;
8   import jakarta.persistence.Column;
9   import jakarta.persistence.Entity;
10  import jakarta.persistence.Id;
11  import jakarta.persistence.JoinColumn;
12  import jakarta.persistence.ManyToOne;
13  import jakarta.persistence.NamedQueries;
14  import jakarta.persistence.NamedQuery;
15  import jakarta.persistence.Table;
16  import jakarta.validation.constraints.NotNull;
17  import jakarta.validation.constraints.Size;
18  import jakarta.xml.bind.annotation.XmlRootElement;
19  import java.io.Serializable;
20
21  /**
22   *
23   * @author erica
24   */
25  @Entity
26  @Table(name = "PessoaFisica")
27  @XmlRootElement
28  @NamedQueries({
29      @NamedQuery(name = "PessoaFisica.findAll", query = "SELECT p FROM PessoaFisica p"),
30      @NamedQuery(name = "PessoaFisica.findByCpf", query = "SELECT p FROM PessoaFisica p WHERE p.cpf = :cpf")})
31  public class PessoaFisica implements Serializable {
32
33      private static final long serialVersionUID = 1L;
34      @Id
35      @Basic(optional = false)
36      @NotNull
37      @Size(min = 1, max = 14)
38      @Column(name = "cpf")
39      private String cpf;
40      @JoinColumn(name = "idPessoa", referencedColumnName = "idPessoa")
41      @ManyToOne
42      private Pessoa idPessoa;
43  }
```

```

42 | private Pessoa idPessoa;
43 |
44 | public PessoaFisica() {
45 | }
46 |
47 | public PessoaFisica(String cpf) {
48 |     this.cpf = cpf;
49 | }
50 |
51 | public String getCpf() {
52 |     return cpf;
53 | }
54 |
55 | public void setCpf(String cpf) {
56 |     this.cpf = cpf;
57 | }
58 |
59 | public Pessoa getIdPessoa() {
60 |     return idPessoa;
61 | }
62 |
63 | public void setIdPessoa(Pessoa idPessoa) {
64 |     this.idPessoa = idPessoa;
65 | }
66 |
67 | @Override
68 | public int hashCode() {
69 |     int hash = 0;
70 |     hash += (cpf != null ? cpf.hashCode() : 0);
71 |     return hash;
72 | }
73 |

```

```

73 |
74 | @Override
75 | public boolean equals(Object object) {
76 |     if (!(object instanceof PessoaFisica)) {
77 |         return false;
78 |     }
79 |     PessoaFisica other = (PessoaFisica) object;
80 |     if ((this.cpf == null && other.cpf != null) || (this.cpf != null && !this.cpf.equals(other.cpf))) {
81 |         return false;
82 |     }
83 |     return true;
84 | }
85 |
86 | @Override
87 | public String toString() {
88 |     return "cadastroee.model.PessoaFisica[ cpf=" + cpf + " ]";
89 | }
90 |
91 | }
92 |

```



```
PessoaFisica.java [-/A] x PessoaJuridica.java [-/A] x
Source History
1  /*
2   * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
3   * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4   */
5   package cadastroee.model;
6
7   import jakarta.persistence.Basic;
8   import jakarta.persistence.Column;
9   import jakarta.persistence.Entity;
10  import jakarta.persistence.Id;
11  import jakarta.persistence.JoinColumn;
12  import jakarta.persistence.ManyToOne;
13  import jakarta.persistence.NamedQueries;
14  import jakarta.persistence.NamedQuery;
15  import jakarta.persistence.Table;
16  import jakarta.validation.constraints.NotNull;
17  import jakarta.validation.constraints.Size;
18  import jakarta.xml.bind.annotation.XmlRootElement;
19  import java.io.Serializable;
20
21  /**
22   *
23   * @author erica
24   */
25  @Entity
26  @Table(name = "PessoaJuridica")
27  @XmlRootElement
28  @NamedQueries({
29      @NamedQuery(name = "PessoaJuridica.findAll", query = "SELECT p FROM PessoaJuridica p"),
30      @NamedQuery(name = "PessoaJuridica.findByCnpj", query = "SELECT p FROM PessoaJuridica p WHERE p.cnpj = :cnpj")})
31  public class PessoaJuridica implements Serializable {
32
33      private static final long serialVersionUID = 1L;
34      @Id
35      @Basic(optional = false)
36      @NotNull
37      @Size(min = 1, max = 18)
38      @Column(name = "cnpj")
39      private String cnpj;
40      @JoinColumn(name = "idPessoa", referencedColumnName = "idPessoa")
41      @ManyToOne
42      private Pessoa idPessoa;
43  }
```

```

43
44 public PessoaJuridica() {
45 }
46
47 public PessoaJuridica(String cnpj) {
48     this.cnpj = cnpj;
49 }
50
51 public String getCnpj() {
52     return cnpj;
53 }
54
55 public void setCnpj(String cnpj) {
56     this.cnpj = cnpj;
57 }
58
59 public Pessoa getIdPessoa() {
60     return idPessoa;
61 }
62
63 public void setIdPessoa(Pessoa idPessoa) {
64     this.idPessoa = idPessoa;
65 }
66
67 @Override
68 public int hashCode() {
69     int hash = 0;
70     hash += (cnpj != null ? cnpj.hashCode() : 0);
71     return hash;
72 }
73

```

```

73
74 @Override
75 public boolean equals(Object object) {
76     if (!(object instanceof PessoaJuridica)) {
77         return false;
78     }
79     PessoaJuridica other = (PessoaJuridica) object;
80     if ((this.cnpj == null && other.cnpj != null) || (this.cnpj != null && !this.cnpj.equals(other.cnpj))) {
81         return false;
82     }
83     return true;
84 }
85
86 @Override
87 public String toString() {
88     return "cadastradoe.model.PessoaJuridica[ cnpj=" + cnpj + " ]";
89 }
90
91 }
92

```

```
Produto.java [-A] x
Source History
1  /*
2   * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
3   * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4   */
5   package cadastroee.model;
6
7   import jakarta.persistence.Basic;
8   import jakarta.persistence.Column;
9   import jakarta.persistence.Entity;
10  import jakarta.persistence.GeneratedValue;
11  import jakarta.persistence.GenerationType;
12  import jakarta.persistence.Id;
13  import jakarta.persistence.NamedQueries;
14  import jakarta.persistence.NamedQuery;
15  import jakarta.persistence.OneToOne;
16  import jakarta.persistence.Table;
17  import jakarta.validation.constraints.Size;
18  import jakarta.xml.bind.annotation.XmlRootElement;
19  import jakarta.xml.bind.annotation.XmlTransient;
20  import java.io.Serializable;
21  import java.util.Collection;
22
23  /**
24   *
25   * @author erica
26   */
27  @Entity
28  @Table(name = "Produto")
29  @XmlRootElement
30  @NamedQueries({
31      @NamedQuery(name = "Produto.findAll", query = "SELECT p FROM Produto p"),
32      @NamedQuery(name = "Produto.findById", query = "SELECT p FROM Produto p WHERE p.id = :id"),
33      @NamedQuery(name = "Produto.findByName", query = "SELECT p FROM Produto p WHERE p.nome = :nome"),
34      @NamedQuery(name = "Produto.findByQuantidade", query = "SELECT p FROM Produto p WHERE p.quantidade = :quantidade"),
35      @NamedQuery(name = "Produto.findByPrecoVenda", query = "SELECT p FROM Produto p WHERE p.precoVenda = :precoVenda")})
36  public class Produto implements Serializable {
37
```

```

36 public class Produto implements Serializable {
37
38     private static final long serialVersionUID = 1L;
39     @Id
40     @GeneratedValue(strategy = GenerationType.IDENTITY)
41     @Basic(optional = false)
42     @Column(name = "id")
43     private Integer id;
44     @Size(max = 255)
45     @Column(name = "nome")
46     private String nome;
47     @Column(name = "quantidade")
48     private Integer quantidade;
49     @Column(name = "precoVenda")
50     private Double precoVenda;
51     @OneToMany(mappedBy = "idProduto")
52     private Collection<Movimento> movimentoCollection;
53
54     public Produto() {
55     }
56
57     public Produto(Integer id) {
58         this.id = id;
59     }
60
61     public Integer getId() {
62         return id;
63     }
64
65     public void setId(Integer id) {
66         this.id = id;
67     }
68
69     public String getNome() {
70         return nome;
71     }
72
73     public void setNome(String nome) {
74         this.nome = nome;
75     }
76

```

```

76
77 public Integer getQuantidade() {
78     return quantidade;
79 }
80
81 public void setQuantidade(Integer quantidade) {
82     this.quantidade = quantidade;
83 }
84
85 public Double getPrecoVenda() {
86     return precoVenda;
87 }
88
89 public void setPrecoVenda(Double precoVenda) {
90     this.precoVenda = precoVenda;
91 }
92
93 @XmlTransient
94 public Collection<Movimento> getMovimentoCollection() {
95     return movimentoCollection;
96 }
97
98 public void setMovimentoCollection(Collection<Movimento> movimentoCollection) {
99     this.movimentoCollection = movimentoCollection;
100 }
101
102 @Override
103 public int hashCode() {
104     int hash = 0;
105     hash += (id != null ? id.hashCode() : 0);
106     return hash;
107 }

```

```

107 }
108
109 @Override
110 public boolean equals(Object object) {
111     if (!(object instanceof Produto)) {
112         return false;
113     }
114     Produto other = (Produto) object;
115     if ((this.id == null && other.id != null) || (this.id != null && !this.id.equals(other.id))) {
116         return false;
117     }
118     return true;
119 }
120
121
122 @Override
123 public String toString() {
124     return "cadastroee.model.Produto[ id=" + id + " ]";
125 }
126
127 }
128

```

```
Usuario.java [-/A] x
Source History
1  /*
2   * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
3   * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4   */
5   package cadastroee.model;
6
7   import jakarta.persistence.Basic;
8   import jakarta.persistence.Column;
9   import jakarta.persistence.Entity;
10  import jakarta.persistence.Id;
11  import jakarta.persistence.NamedQueries;
12  import jakarta.persistence.NamedQuery;
13  import jakarta.persistence.OneToOne;
14  import jakarta.persistence.Table;
15  import jakarta.validation.constraints.NotNull;
16  import jakarta.validation.constraints.Size;
17  import jakarta.xml.bind.annotation.XmlRootElement;
18  import jakarta.xml.bind.annotation.XmlTransient;
19  import java.io.Serializable;
20  import java.util.Collection;
21
22  /**
23   *
24   * @author erica
25   */
26  @Entity
27  @Table(name = "Usuario")
28  @XmlRootElement
29  @NamedQueries({
30      @NamedQuery(name = "Usuario.findAll", query = "SELECT u FROM Usuario u"),
31      @NamedQuery(name = "Usuario.findByLogin", query = "SELECT u FROM Usuario u WHERE u.login = :login"),
32      @NamedQuery(name = "Usuario.findBySenha", query = "SELECT u FROM Usuario u WHERE u.senha = :senha")}
33  )
34  public class Usuario implements Serializable {
```

```
34
35     private static final long serialVersionUID = 1L;
36     @Id
37     @Basic(optional = false)
38     @NotNull
39     @Size(min = 1, max = 20)
40     @Column(name = "login")
41     private String login;
42     @Size(max = 20)
43     @Column(name = "senha")
44     private String senha;
45     @OneToMany(mappedBy = "idUsuario")
46     private Collection<Movimento> movimentoCollection;
47
48     public Usuario() {
49     }
50
51     public Usuario(String login) {
52         this.login = login;
53     }
54
55     public String getLogin() {
56         return login;
57     }
58
59     public void setLogin(String login) {
60         this.login = login;
61     }
62
63     public String getSenha() {
64         return senha;
65     }
66
67     public void setSenha(String senha) {
68         this.senha = senha;
69     }
70
```

```

70
71 @XmlTransient
72 public Collection<Movimento> getMovimentoCollection() {
73     return movimentoCollection;
74 }
75
76 public void setMovimentoCollection(Collection<Movimento> movimentoCollection) {
77     this.movimentoCollection = movimentoCollection;
78 }
79
80 @Override
81 public int hashCode() {
82     int hash = 0;
83     hash += (login != null ? login.hashCode() : 0);
84     return hash;
85 }
86
87 @Override
88 public boolean equals(Object object) {
89     if (!(object instanceof Usuario)) {
90         return false;
91     }
92     Usuario other = (Usuario) object;
93     if ((this.login == null && other.login != null) || (this.login != null && !this.login.equals(other.login))) {
94         return false;
95     }
96     return true;
97 }
98
99 @Override
100 public String toString() {
101     return "cadastrado.model.Usuario[ login=" + login + " ]";
102 }
103
104 }
105

```



```
AbstractFacade.java [-A] x
Source History
1  /**
2   * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
3   * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4   */
5   package cadastroee.controller;
6
7   import jakarta.persistence.EntityManager;
8   import java.util.List;
9
10  /**
11   *
12   * @author erica
13   */
14  public abstract class AbstractFacade<T> {
15
16      private Class<T> entityClass;
17
18      public AbstractFacade(Class<T> entityClass) {
19          this.entityClass = entityClass;
20      }
21
22      protected abstract EntityManager getEntityManager();
23
24      public void create(T entity) {
25          getEntityManager().persist(entity);
26      }
27
28      public void edit(T entity) {
29          getEntityManager().merge(entity);
30      }
31
32      public void remove(T entity) {
33          getEntityManager().remove(getEntityManager().merge(entity));
34      }
35
36      public T find(Object id) {
37          return getEntityManager().find(entityClass, id);
38      }
39
40      public List<T> findAll() {
41          jakarta.persistence.criteria.CriteriaQuery cq = getEntityManager().getCriteriaBuilder().createQuery();
42          cq.select(cq.from(entityClass));
43          return getEntityManager().createQuery(cq).getResultList();
44      }
45
46      public List<T> findRange(int[] range) {
47          jakarta.persistence.criteria.CriteriaQuery cq = getEntityManager().getCriteriaBuilder().createQuery();
48          cq.select(cq.from(entityClass));
49          jakarta.persistence.Query q = getEntityManager().createQuery(cq);
50          q.setMaxResults(range[1] - range[0] + 1);
51          q.setFirstResult(range[0]);
52          return q.getResultList();
53      }
54
55      public int count() {
56          jakarta.persistence.criteria.CriteriaQuery cq = getEntityManager().getCriteriaBuilder().createQuery();
57          jakarta.persistence.criteria.Root<T> rt = cq.from(entityClass);
58          cq.select(getEntityManager().getCriteriaBuilder().count(rt));
59          jakarta.persistence.Query q = getEntityManager().createQuery(cq);
60          return ((Long) q.getSingleResult()).intValue();
61      }
62  }
63
64  }
```

```
MovimentoFacade.java [-/A] x
Source History
1  /*
2   * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
3   * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4   */
5   package cadastroee.controller;
6
7   import cadastroee.model.Movimento;
8   import jakarta.ejb.Stateless;
9   import jakarta.persistence.EntityManager;
10  import jakarta.persistence.PersistenceContext;
11
12  /**
13   *
14   * @author erica
15   */
16  @Stateless
17  public class MovimentoFacade extends AbstractFacade<Movimento> {
18
19      @PersistenceContext(unitName = "CadastroEE-ejbPU")
20      private EntityManager em;
21
22      @Override
23      protected EntityManager getEntityManager() {
24          return em;
25      }
26
27      public MovimentoFacade() {
28          super(Movimento.class);
29      }
30
31  }
32
```

```
PessoaFacade.java [-/A] x
Source History
1  /*
2   * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
3   * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4   */
5   package cadastroee.controller;
6
7   import cadastroee.model.Pessoa;
8   import jakarta.ejb.Stateless;
9   import jakarta.persistence.EntityManager;
10  import jakarta.persistence.PersistenceContext;
11
12  /**
13   *
14   * @author erica
15   */
16  @Stateless
17  public class PessoaFacade extends AbstractFacade<Pessoa> {
18
19      @PersistenceContext(unitName = "CadastroEE-ejbPU")
20      private EntityManager em;
21
22      @Override
23      protected EntityManager getEntityManager() {
24          return em;
25      }
26
27      public PessoaFacade() {
28          super(Pessoa.class);
29      }
30
31  }
32
```

```
PessoaFisicaFacade.java [-/A] x
Source History
1  /*
2   * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
3   * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4   */
5   package cadastroee.controller;
6
7   import cadastroee.model.PessoaFisica;
8   import jakarta.ejb.Stateless;
9   import jakarta.persistence.EntityManager;
10  import jakarta.persistence.PersistenceContext;
11
12  /**
13   *
14   * @author erica
15   */
16  @Stateless
17  public class PessoaFisicaFacade extends AbstractFacade<PessoaFisica> {
18
19      @PersistenceContext(unitName = "CadastroEE-ejbPU")
20      private EntityManager em;
21
22      @Override
23      protected EntityManager getEntityManager() {
24          return em;
25      }
26
27      public PessoaFisicaFacade() {
28          super(PessoaFisica.class);
29      }
30
31  }
32
```

```
PessoaFisicaFacade.java [-/A] x PessoaJuridicaFacade.java [-/A] x
Source History
1  /*
2   * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
3   * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4   */
5   package cadastroee.controller;
6
7   import cadastroee.model.PessoaJuridica;
8   import jakarta.ejb.Stateless;
9   import jakarta.persistence.EntityManager;
10  import jakarta.persistence.PersistenceContext;
11
12  /**
13   *
14   * @author erica
15   */
16  @Stateless
17  public class PessoaJuridicaFacade extends AbstractFacade<PessoaJuridica> {
18
19      @PersistenceContext(unitName = "CadastroEE-ejbPU")
20      private EntityManager em;
21
22      @Override
23      protected EntityManager getEntityManager() {
24          return em;
25      }
26
27      public PessoaJuridicaFacade() {
28          super(PessoaJuridica.class);
29      }
30
31  }
32
```

```
ProdutoFacade.java [-/A] x
Source History
1  /**
2   * Click nbfs:///nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
3   * Click nbfs:///nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4   */
5   package cadastroee.controller;
6
7   import cadastroee.model.Produto;
8   import jakarta.ejb.Stateless;
9   import jakarta.persistence.EntityManager;
10  import jakarta.persistence.PersistenceContext;
11
12  /**
13   *
14   * @author erica
15   */
16  @Stateless
17  public class ProdutoFacade extends AbstractFacade<Produto> {
18
19      @PersistenceContext(unitName = "CadastroEE-ejbPU")
20      private EntityManager em;
21
22      @Override
23      protected EntityManager getEntityManager() {
24          return em;
25      }
26
27      public ProdutoFacade() {
28          super(Produto.class);
29      }
30
31  }
32
```

```
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package cadastroee.controller;

import cadastroee.model.Usuario;
import jakarta.ejb.Stateless;
import jakarta.persistence.EntityManager;
import jakarta.persistence.PersistenceContext;

/**
 *
 * @author erica
 */
@Stateless
public class UsuarioFacade extends AbstractFacade<Usuario> {

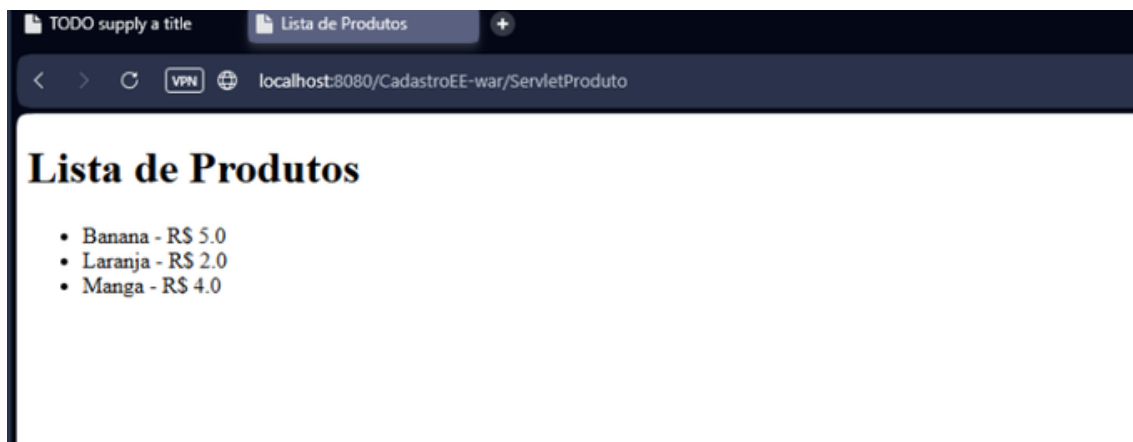
    @PersistenceContext(unitName = "CadastroEE-ejbPU")
    private EntityManager em;

    @Override
    protected EntityManager getEntityManager() {
        return em;
    }

    public UsuarioFacade() {
        super(Usuario.class);
    }

}
```

```
Output x
GlassFish Server x CadastroEE (run) x
CadastroEE-war.compile:
CadastroEE-war.compile-jsp:
CadastroEE-war.do-ear-dist:
Building jar: E:\Estacio\CadastroEE\CadastroEE-war\dist\CadastroEE-war.war
CadastroEE-war.dist-ear:
pre-pre-compile:
pre-compile:
Copying 1 file to E:\Estacio\CadastroEE\build
Copying 1 file to E:\Estacio\CadastroEE\build
do-compile:
post-compile:
compile:
pre-dist:
post-dist:
dist-directory-deploy:
pre-run-deploy:
Initial deploying CadastroEE to E:\Estacio\CadastroEE\dist\gfdeploy\CadastroEE
Completed initial distribution of CadastroEE
post-run-deploy:
run-deploy:
Browsing: http://localhost:8080/CadastroEE-war
run-display-browser:
run-ac:
run:
BUILD SUCCESSFUL (total time: 7 seconds)
```



a. Como é organizado um projeto corporativo no NetBeans?

No NetBeans, aplicações corporativas costumam ser estruturadas por meio de um projeto do tipo Enterprise Application (EAR), responsável por agrupar diferentes módulos da aplicação. Normalmente, existe um módulo EJB, encarregado das regras de negócio e da persistência de dados, e um módulo Web, destinado à interface gráfica e ao atendimento das requisições dos usuários. Essa divisão favorece a organização do código, facilita a manutenção e segue práticas amplamente utilizadas no desenvolvimento de sistemas corporativos.

b. Qual o papel das tecnologias JPA e EJB na construção de um aplicativo para a plataforma Web no ambiente Java?

As tecnologias JPA e EJB possuem funções complementares dentro de uma aplicação Java Web. A JPA é utilizada para realizar o mapeamento entre objetos Java e tabelas do banco de dados, permitindo que as operações de persistência sejam realizadas de forma mais simples e intuitiva. Já os EJBs são responsáveis por concentrar a lógica de negócio da aplicação, oferecendo recursos como gerenciamento de transações, controle de acesso e integração entre os componentes do sistema.

c. Como o NetBeans viabiliza a melhoria de produtividade ao lidar com as tecnologias JPA e EJB?

O NetBeans oferece diversos recursos que agilizam o desenvolvimento de aplicações corporativas. Entre eles estão os assistentes para geração automática de entidades a partir do banco de dados e a criação de Session Beans com operações básicas de manipulação de dados. Essas funcionalidades reduzem a necessidade de escrever código repetitivo, diminuem a ocorrência de erros e tornam o processo de desenvolvimento mais rápido e eficiente.

d. O que são Servlets e como o NetBeans oferece suporte à construção desse tipo de componente em um projeto Web?

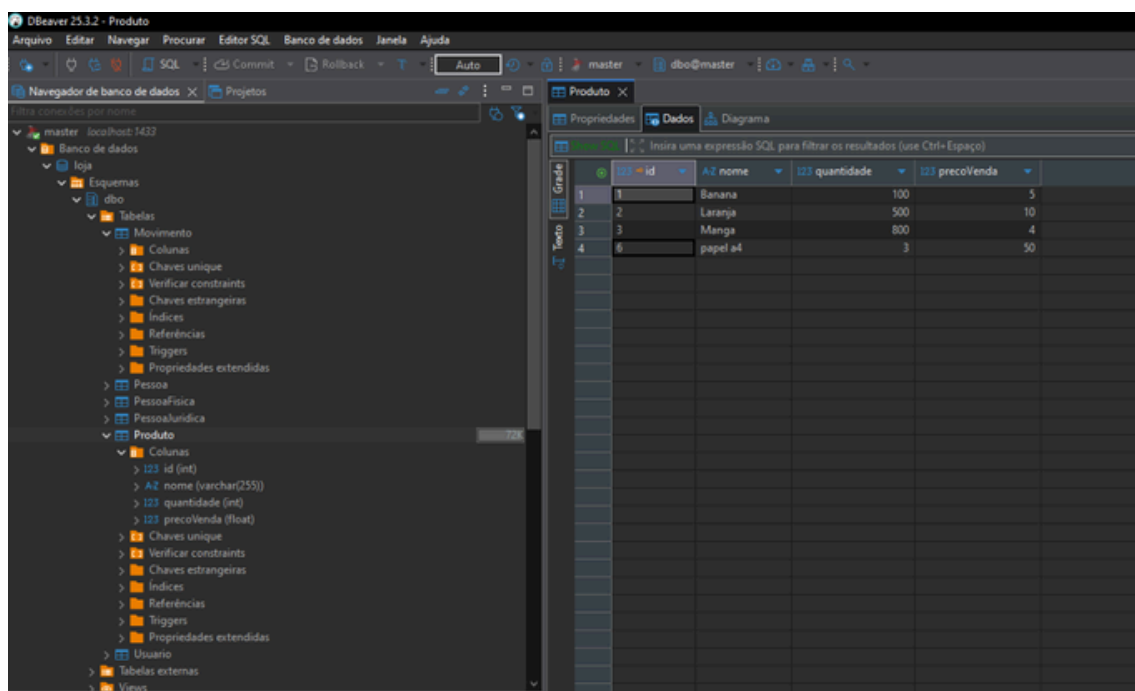
Servlets são componentes Java responsáveis por receber, processar e responder às requisições enviadas pelos usuários através do navegador. Eles atuam como intermediários entre a interface da aplicação e as camadas de negócio e persistência. O NetBeans

simplifica a criação desses componentes ao disponibilizar assistentes de geração automática, configuração de mapeamentos e integração com servidores de aplicação, facilitando o desenvolvimento de aplicações Web.

e. Como é feita a comunicação entre os Servlets e os Session Beans do pool de EJBs?

A interação entre Servlets e Session Beans ocorre por meio da injeção de dependência disponibilizada pela plataforma Java EE. Utilizando a anotação `@EJB`, o desenvolvedor pode solicitar uma referência para um Session Bean diretamente dentro do Servlet. Dessa forma, o servidor de aplicação fornece automaticamente uma instância gerenciada do componente, permitindo o acesso às regras de negócio sem a necessidade de criar ou gerenciar objetos manualmente.

Banco de Dados

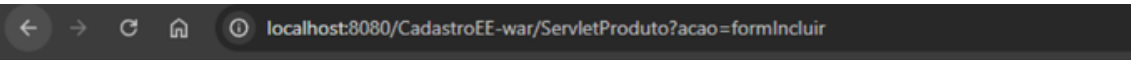


The screenshot shows the DBeaver 25.3.2 interface. On the left, the 'Navegador de banco de dados' (Database Navigator) displays a tree structure for a database named 'master' on 'localhost:1433'. The tree is expanded to show the 'Produto' table under the 'dbo' schema. The table structure is as follows:

id	nome	quantidade	precoVenda
1	Banana	100	5
2	Laranja	500	10
3	Manga	800	4
4	papel a4	3	50

The right pane shows the 'Dados' (Data) tab for the 'Produto' table, displaying the same data as the table structure. The top toolbar includes options for 'Arquivo', 'Editar', 'Navegar', 'Procurar', 'Editor SQL', 'Banco de dados', 'Janela', and 'Ajuda'.

Aplicação

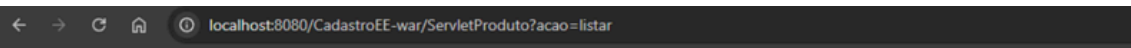


Dados do Produto

Nome:

Quantidade:

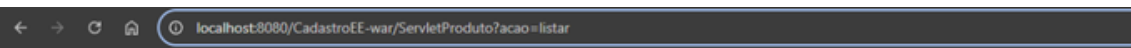
Preço de Venda:



Lista de Produtos Cadastrados

ID	Nome	Preço de Venda
1	Banana	R\$ 5.0
2	Laranja	R\$ 2.0
3	Manga	R\$ 4.0
4	caju	R\$ 300.0
5	atum	R\$ 7.0

[Novo Produto](#)
[Voltar para Início](#)



Lista de Produtos Cadastrados

ID	Nome	Preço de Venda	Ações
1	Banana	R\$ 5.0	Alterar Excluir
2	Laranja	R\$ 2.0	Alterar Excluir
3	Manga	R\$ 4.0	Alterar Excluir
4	caju	R\$ 300.0	Alterar Excluir
5	atum	R\$ 7.0	Alterar Excluir

[Novo Produto](#)
[Voltar para Início](#)



Lista de Produtos Cadastrados

ID	Nome	Preço de Venda	Ações
1	Banana	R\$ 5.0	Alterar Excluir
2	Laranja	R\$ 2.0	Alterar Excluir
3	Manga	R\$ 4.0	Alterar Excluir
5	atum	R\$ 7.0	Alterar Excluir

[Novo Produto](#)
[Voltar para Início](#)



Lista de Produtos Cadastrados

ID	Nome	Preço de Venda	Ações
1	Banana	R\$ 5.0	Alterar Excluir
2	Laranja	R\$ 10.0	Alterar Excluir
3	Manga	R\$ 4.0	Alterar Excluir
5	atum	R\$ 7.0	Alterar Excluir

[Novo Produto](#)
[Voltar para Início](#)

a. Como funciona o padrão Front Controller, e como ele é implementado em um aplicativo Web Java, na arquitetura MVC?

O padrão Front Controller consiste em centralizar o tratamento das requisições dos usuários em um único componente responsável por controlar o fluxo da aplicação. Em vez de cada página ou funcionalidade possuir seu próprio controlador, todas as solicitações passam inicialmente por um único Servlet, que analisa a ação desejada e direciona o processamento adequado.

Na arquitetura MVC, esse controlador atua como intermediário entre a camada de visualização (View) e a camada de negócio (Model). Após receber uma requisição, o Front Controller executa as operações necessárias, consulta ou altera dados por meio das regras de negócio e encaminha o usuário para a página apropriada. Essa abordagem contribui para uma melhor organização do código, reduz a duplicação de lógica e facilita a manutenção do sistema.

b. Quais as diferenças e semelhanças entre Servlets e JSPs?

As tecnologias Servlet e JSP (JavaServer Pages) constituem os pilares da camada Web na plataforma Java EE, compartilhando a finalidade comum de gerar conteúdo dinâmico para o cliente. A principal semelhança reside no fato de que o container Web, como o GlassFish, converte automaticamente todo arquivo JSP em um Servlet antes de sua execução, garantindo que ambos tenham acesso aos mesmos objetos implícitos da especificação, como requisição e sessão HTTP.

No entanto, as tecnologias possuem focos arquiteturais distintos para atender ao padrão MVC. O Servlet é uma classe Java pura, sendo o componente ideal para a

implementação da lógica de controle, processamento de dados e tomada de decisão. Inversamente, o JSP é um arquivo baseado em texto que prioriza o código HTML, permitindo a inserção de lógica de apresentação através de tags JSTL. Dessa forma, evita-se a má prática de inserir código de formatação visual dentro de classes Java, o que dificultaria a manutenção do software.

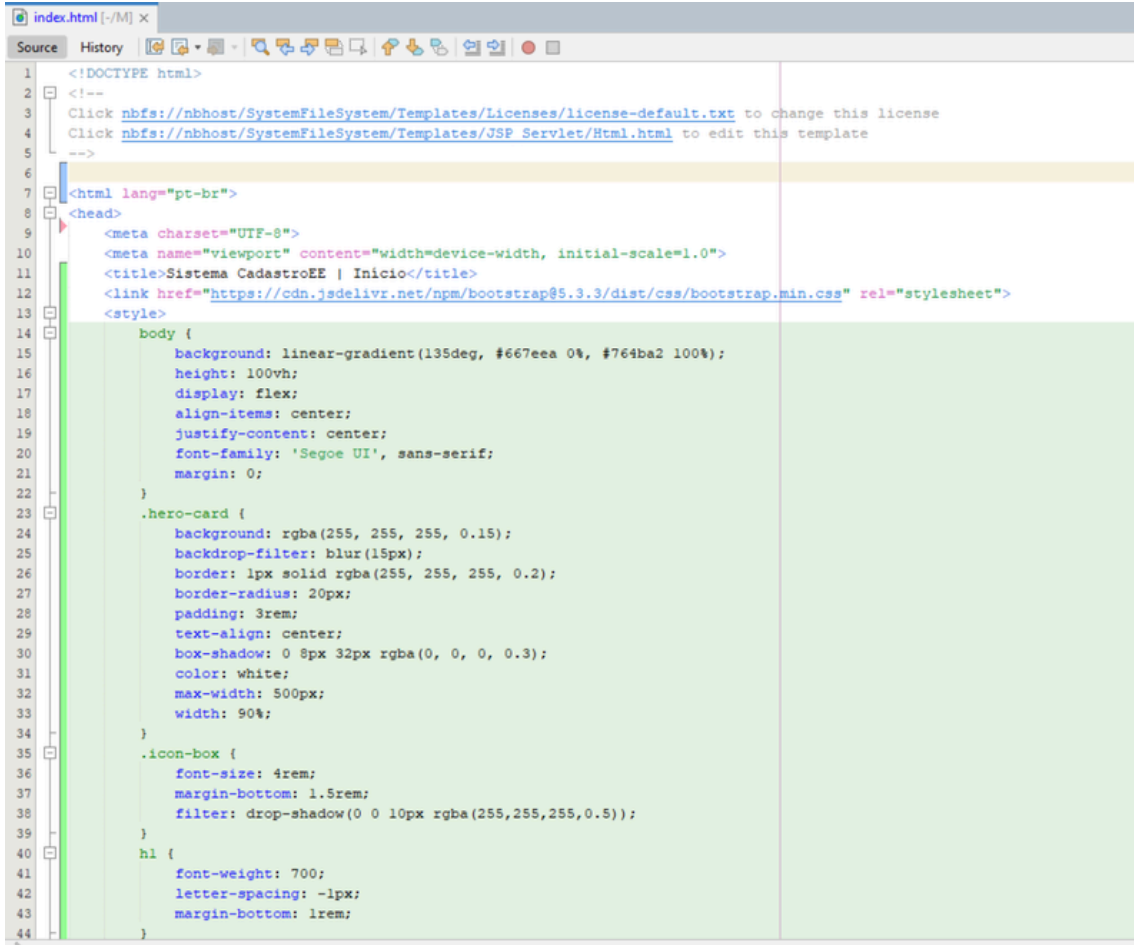
c. Qual a diferença entre um redirecionamento simples e o uso do método forward, a partir do RequestDispatcher? Para que servem parâmetros e atributos nos objetos HttpRequest?

A navegação entre os componentes de um aplicativo Java Web pode ser realizada via redirecionamento (Redirect) ou pelo método Forward do RequestDispatcher, cada qual com impactos distintos no gerenciamento de dados. O Forward ocorre inteiramente no lado do servidor, onde o Servlet encaminha a execução para um JSP sem que o navegador do cliente tenha ciência da mudança. Esse mecanismo é mais eficiente e preserva os atributos da requisição original, sendo a escolha adequada para carregar a lista de produtos exibida ao usuário.

Já o redirecionamento (sendRedirect) instrui o navegador a realizar uma nova requisição para uma URL distinta, o que resulta na limpeza de todos os dados da requisição anterior. Esta abordagem é fundamental após operações de escrita, como a inclusão ou alteração de registros, pois impede que o usuário submeta dados duplicados ao atualizar a página (F5).

No que tange ao tráfego de informações, os parâmetros consistem em dados enviados pelo cliente ao servidor através de formulários ou URLs, apresentando-se sempre no formato de texto. Em contrapartida, os atributos são objetos Java complexos armazenados temporariamente na requisição pelo servidor, permitindo que o Servlet compartilhe coleções de dados, como listas de entidades, diretamente com as páginas JSP para renderização.

Design da Interface



```
1 <!DOCTYPE html>
2 <!--
3 Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
4 Click nbfs://nbhost/SystemFileSystem/Templates/JSP\_Servlet/Html.html to edit this template
5 -->
6
7 <html lang="pt-br">
8 <head>
9   <meta charset="UTF-8">
10  <meta name="viewport" content="width=device-width, initial-scale=1.0">
11  <title>Sistema CadastroEE | Inicio</title>
12  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
13  <style>
14    body {
15      background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
16      height: 100vh;
17      display: flex;
18      align-items: center;
19      justify-content: center;
20      font-family: 'Segoe UI', sans-serif;
21      margin: 0;
22    }
23    .hero-card {
24      background: rgba(255, 255, 255, 0.15);
25      backdrop-filter: blur(15px);
26      border: 1px solid rgba(255, 255, 255, 0.2);
27      border-radius: 20px;
28      padding: 3rem;
29      text-align: center;
30      box-shadow: 0 8px 32px rgba(0, 0, 0, 0.3);
31      color: white;
32      max-width: 500px;
33      width: 90%;
34    }
35    .icon-box {
36      font-size: 4rem;
37      margin-bottom: 1.5rem;
38      filter: drop-shadow(0 0 10px rgba(255,255,255,0.5));
39    }
40    h1 {
41      font-weight: 700;
42      letter-spacing: -1px;
43      margin-bottom: 1rem;
44    }
```

```

44     }
45     p {
46         opacity: 0.9;
47         font-size: 1.1rem;
48         margin-bottom: 2rem;
49     }
50     .btn-start {
51         background-color: #fff;
52         color: #764ba2;
53         font-weight: 600;
54         padding: 12px 35px;
55         border-radius: 50px;
56         text-transform: uppercase;
57         letter-spacing: 1px;
58         transition: all 0.3s ease;
59         text-decoration: none;
60         display: inline-block;
61         box-shadow: 0 4px 15px rgba(0,0,0,0.2);
62     }
63     .btn-start:hover {
64         transform: translateY(-3px);
65         box-shadow: 0 6px 20px rgba(0,0,0,0.3);
66         background-color: #f8f9fa;
67         color: #667eea;
68     }
69 </style>
70 </head>
71 <body>
72
73     <div class="hero-card">
74         <div class="icon-box">📦</div>
75         <h1>CadastroEE</h1>
76         <p>Gestão Inteligente de Produtos e Movimentações com Java Enterprise Edition.</p>
77         <hr class="my-4" style="opacity: 0.2;">
78         <a href="ServletProduto" class="btn-start">Acessar Sistema</a>
79     </div>
80
81     <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
82 </body>
83 </html>

```

```
ManterProduto.jsp [-/A] x
Source History
Document : ManterProduto
Created on : 14 de jan. de 2026, 17:48:00
Author : erica
-->
<!--
2
3
4
5
6
7 <%%page contentType="text/html" pageEncoding="UTF-8"%>
8 <!DOCTYPE html>
9 <html>
10 <head>
11 <title>Manter Produto</title>
12 <meta charset="UTF-8">
13 <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="style:
14 <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></sc:
15 </head>
16 <body class="container">
17 <h1 class="my-4">Dados do Produto</h1>
18
19 <form action="ServletProduto" method="post" class="card p-4 shadow-sm">
20 <input type="hidden" name="acao" value="${produto == null ? 'incluir' : 'alterar'}">
21 <input type="hidden" name="id" value="${produto.id}">
22
23 <div class="mb-3">
24 <label class="form-label">Nome do Produto</label>
25 <input type="text" name="nome" class="form-control" value="${p.nome}" required>
26 </div>
27
28 <div class="mb-3">
29 <label class="form-label">Quantidade</label>
30 <input type="number" name="quantidade" class="form-control" value="${p.quantidade}" require:
31 </div>
32
33 <div class="mb-3">
34 <label class="form-label">Preço de Venda</label>
35 <input type="text" name="precoVenda" class="form-control" value="${p.precoVenda}" required>
36 </div>
37
38 <div class="mt-4">
39 <button type="submit" class="btn btn-primary">Salvar Produto</button>
40 <a href="ServletProduto?acao=listar" class="btn btn-secondary">Cancelar</a>
41 </div>
42 </form>
43 </body>
44 </html>
```

```
ListaProdutos.jsp [-A] X
Source History
<!DOCTYPE html>
<html lang="pt-br">
  <head>
    <meta charset="UTF-8">
    <title>Sistema CadastroEE - Lista</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
  </head>
  <body>
    <div class="container">
      <h1>Lista de Produtos</h1>
      <table>
        <thead>
          <tr>
            <th>ID</th>
            <th>Nome</th>
            <th>Preço</th>
            <th>Ações</th>
          </tr>
        </thead>
        <tbody>
          <tr>
            <td>1</td>
            <td>Produto A</td>
            <td>R$ 10,00</td>
            <td><button class="btn btn-primary">Alterar</button> <button class="btn btn-danger">Excluir</button></td>
          </tr>
          <tr>
            <td>2</td>
            <td>Produto B</td>
            <td>R$ 20,00</td>
            <td><button class="btn btn-primary">Alterar</button> <button class="btn btn-danger">Excluir</button></td>
          </tr>
          <tr>
            <td>3</td>
            <td>Produto C</td>
            <td>R$ 30,00</td>
            <td><button class="btn btn-primary">Alterar</button> <button class="btn btn-danger">Excluir</button></td>
          </tr>
          <tr>
            <td>4</td>
            <td>Produto D</td>
            <td>R$ 40,00</td>
            <td><button class="btn btn-primary">Alterar</button> <button class="btn btn-danger">Excluir</button></td>
          </tr>
          <tr>
            <td>5</td>
            <td>Produto E</td>
            <td>R$ 50,00</td>
            <td><button class="btn btn-primary">Alterar</button> <button class="btn btn-danger">Excluir</button></td>
          </tr>
          <tr>
            <td>6</td>
            <td>Produto F</td>
            <td>R$ 60,00</td>
            <td><button class="btn btn-primary">Alterar</button> <button class="btn btn-danger">Excluir</button></td>
          </tr>
          <tr>
            <td>7</td>
            <td>Produto G</td>
            <td>R$ 70,00</td>
            <td><button class="btn btn-primary">Alterar</button> <button class="btn btn-danger">Excluir</button></td>
          </tr>
          <tr>
            <td>8</td>
            <td>Produto H</td>
            <td>R$ 80,00</td>
            <td><button class="btn btn-primary">Alterar</button> <button class="btn btn-danger">Excluir</button></td>
          </tr>
          <tr>
            <td>9</td>
            <td>Produto I</td>
            <td>R$ 90,00</td>
            <td><button class="btn btn-primary">Alterar</button> <button class="btn btn-danger">Excluir</button></td>
          </tr>
          <tr>
            <td>10</td>
            <td>Produto J</td>
            <td>R$ 100,00</td>
            <td><button class="btn btn-primary">Alterar</button> <button class="btn btn-danger">Excluir</button></td>
          </tr>
        </tbody>
      </table>
      <div class="text-center">
        <button class="btn btn-primary">Adicionar Novo Produto</button>
      </div>
    </div>
  </body>
</html>
```

```

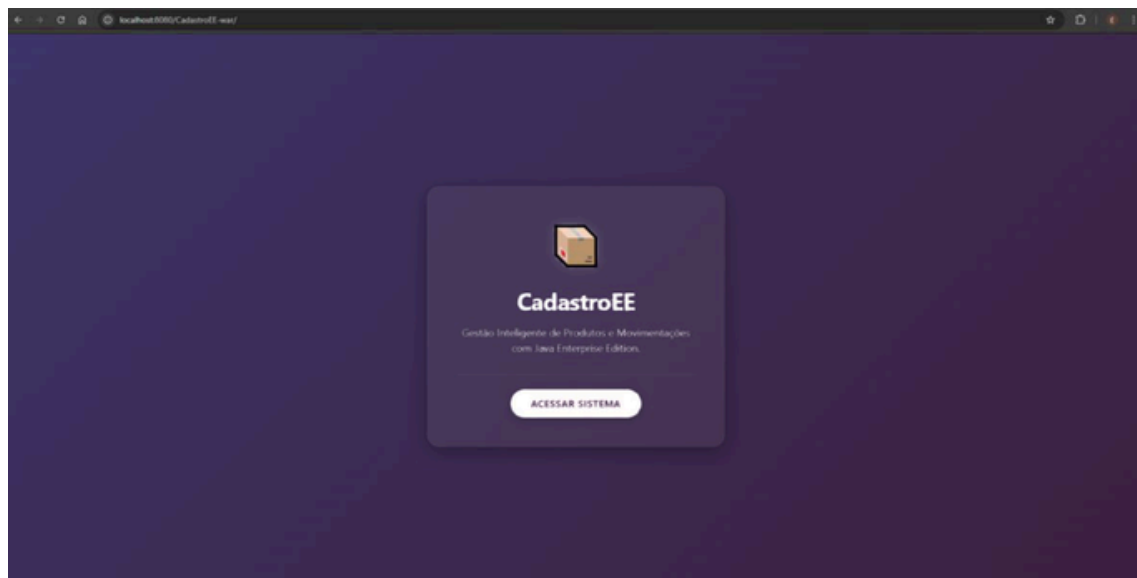
    <tr>
      <td>1</td>
      <td>Produto A</td>
      <td>R$ 10,00</td>
      <td><button class="btn btn-primary">Alterar</button> <button class="btn btn-danger">Excluir</button></td>
    </tr>
    <tr>
      <td>2</td>
      <td>Produto B</td>
      <td>R$ 20,00</td>
      <td><button class="btn btn-primary">Alterar</button> <button class="btn btn-danger">Excluir</button></td>
    </tr>
    <tr>
      <td>3</td>
      <td>Produto C</td>
      <td>R$ 30,00</td>
      <td><button class="btn btn-primary">Alterar</button> <button class="btn btn-danger">Excluir</button></td>
    </tr>
    <tr>
      <td>4</td>
      <td>Produto D</td>
      <td>R$ 40,00</td>
      <td><button class="btn btn-primary">Alterar</button> <button class="btn btn-danger">Excluir</button></td>
    </tr>
    <tr>
      <td>5</td>
      <td>Produto E</td>
      <td>R$ 50,00</td>
      <td><button class="btn btn-primary">Alterar</button> <button class="btn btn-danger">Excluir</button></td>
    </tr>
    <tr>
      <td>6</td>
      <td>Produto F</td>
      <td>R$ 60,00</td>
      <td><button class="btn btn-primary">Alterar</button> <button class="btn btn-danger">Excluir</button></td>
    </tr>
    <tr>
      <td>7</td>
      <td>Produto G</td>
      <td>R$ 70,00</td>
      <td><button class="btn btn-primary">Alterar</button> <button class="btn btn-danger">Excluir</button></td>
    </tr>
    <tr>
      <td>8</td>
      <td>Produto H</td>
      <td>R$ 80,00</td>
      <td><button class="btn btn-primary">Alterar</button> <button class="btn btn-danger">Excluir</button></td>
    </tr>
    <tr>
      <td>9</td>
      <td>Produto I</td>
      <td>R$ 90,00</td>
      <td><button class="btn btn-primary">Alterar</button> <button class="btn btn-danger">Excluir</button></td>
    </tr>
    <tr>
      <td>10</td>
      <td>Produto J</td>
      <td>R$ 100,00</td>
      <td><button class="btn btn-primary">Alterar</button> <button class="btn btn-danger">Excluir</button></td>
    </tr>
  </tbody>
</table>
<div class="text-center">
  <button class="btn btn-primary">Adicionar Novo Produto</button>
</div>
</div>
</body>
</html>
```

```

67 </style>
68 </head>
69 <body class="container">
70 <h1 class="my-4">Lista de Produtos Cadastrados</h1>
71
72 <table class="table table-striped table-hover shadow-sm">
73 <thead class="table-dark">
74 <tr>
75 <th>ID</th>
76 <th>Nome</th>
77 <th>Preço de Venda</th>
78 <th>Ações</th>
79 </tr>
80 </thead>
81 <tbody>
82 <forEach var="p" items="${produtos}">
83 <tr>
84 <td>${p.id}</td>
85 <td>${p.nome}</td>
86 <td>R$ ${p.precoVenda}</td>
87 <td>
88 <a href="ServletProduto?acao=formAlterar&id=${p.id}" class="btn btn-primary btn-sm">Alterar</a>
89 <a href="ServletProduto?acao=excluir&id=${p.id}" class="btn btn-danger btn-sm" onclick="return confirm('Excluir?');">Excluir</a>
90 </td>
91 </tr>
92 </forEach>
93 </tbody>
94 </table>
95
96 <a href="ServletProduto?acao=formIncluir" class="btn btn-primary m-2">Novo Produto</a>
97 <br>
98 <a href="index.html" class="ms-2">Voltar para Inicio</a>
99 </body>
100 </html>

```

Tela inicial do programa



Tela de lista de produtos

localhost:8080/CadastroEE_war/ServletProduto

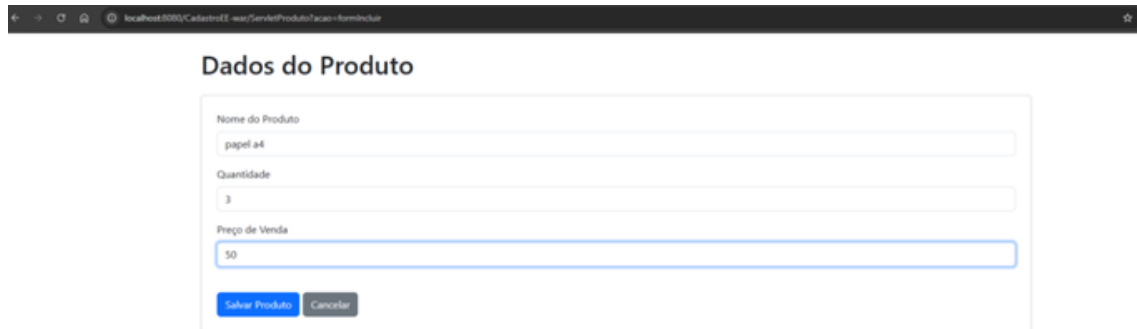
Lista de Produtos Cadastrados

ID	NOME	PREÇO DE VENDA	AÇÕES
1	Banana	R\$ 5.0	Alterar Excluir
2	Laranja	R\$ 10.0	Alterar Excluir
3	Manga	R\$ 4.0	Alterar Excluir
6	papel a4	R\$ 50.0	Alterar Excluir

[Novo Produto](#)

[Voltar para Inicio](#)

Tela de Adição de produtos



Dados do Produto

Nome do Produto
papel a4

Quantidade
3

Preço de Venda
50

Salvar Produto Cancelar

a. Como o framework Bootstrap é utilizado?

O Bootstrap é um framework amplamente utilizado no desenvolvimento de interfaces web por disponibilizar diversos componentes visuais prontos para uso, contribuindo para a criação de páginas mais organizadas e modernas. Sua utilização pode ser realizada através da inclusão de arquivos CSS e JavaScript, seja por meio de CDN ou de arquivos locais vinculados ao projeto. No sistema desenvolvido, o framework foi empregado na estilização de elementos como tabelas, formulários e botões, proporcionando uma aparência mais profissional e agradável ao usuário, além de reduzir a necessidade de criar estilos CSS manualmente.

b. Por que o Bootstrap garante a independência estrutural do HTML?

O Bootstrap contribui para a separação entre a estrutura da página e sua aparência visual, permitindo que o conteúdo HTML permaneça organizado enquanto a estilização é aplicada por meio de classes CSS fornecidas pelo framework. Dessa forma, alterações relacionadas ao layout, cores, espaçamentos e demais aspectos visuais podem ser realizadas sem a necessidade de modificar a estrutura principal do documento. Essa abordagem facilita a manutenção e a atualização da interface, tornando o desenvolvimento mais prático e preservando a organização e a semântica do código HTML.

c. Qual a relação entre o Bootstrap e a responsividade da página?

A responsividade é uma das principais características do Bootstrap, já que o framework foi desenvolvido para adaptar interfaces a diferentes tamanhos de tela. Para isso, ele utiliza um sistema de grades responsivas e pontos de quebra (breakpoints), que permitem reorganizar automaticamente os elementos da página conforme o dispositivo utilizado pelo usuário. Dessa forma, componentes como tabelas, formulários e botões podem ser exibidos de maneira adequada tanto em computadores quanto em smartphones e tablets. No sistema desenvolvido, essa funcionalidade contribui para uma melhor experiência de navegação, garantindo que as informações permaneçam acessíveis e organizadas independentemente da resolução da tela.

Conclusão

O desenvolvimento desta atividade permitiu colocar em prática diversos conceitos relacionados à construção de aplicações corporativas na plataforma Java. Durante a implementação do sistema, foi possível compreender como diferentes tecnologias trabalham em conjunto para formar uma aplicação completa, desde o armazenamento das informações em banco de dados até a apresentação dos dados ao usuário por meio de uma interface Web.

A utilização de JPA, EJB, Servlets e JSP demonstrou a importância da divisão de responsabilidades entre os componentes do sistema, tornando o projeto mais organizado e facilitando futuras modificações e manutenções. Essa organização também contribui para a reutilização de código e para a construção de aplicações mais estruturadas e escaláveis.

Outro aspecto relevante observado durante a prática foi a utilização do Bootstrap para aprimorar a experiência do usuário. A aplicação do framework possibilitou a criação de páginas mais intuitivas, com melhor apresentação visual e adaptação a diferentes dispositivos, sem a necessidade de um grande volume de código CSS personalizado.

De modo geral, a atividade proporcionou uma visão mais ampla sobre o funcionamento de aplicações corporativas baseadas em Java EE, reforçando conhecimentos sobre persistência de dados, desenvolvimento Web e integração entre camadas. Os resultados obtidos demonstram a eficácia da arquitetura utilizada e evidenciam a importância dessas tecnologias no desenvolvimento de sistemas utilizados em ambientes corporativos.



Estácio

Trabalho Prático | DGT2811 Desenvolvimento Back-End
Corporativo com Java E Cloud

Aluno: Adrian Guilherme Grangeiro Cavalcante
Matricula: 202402131931

Polo: Porto Velho - RO